

Salesforce Project Documentation

WhatsNext Vision Motors CRM

Contents

1	Project Overview	2
2	Objectives and Scope	2
2.1	Primary Business Objectives	2
2.2	Project Scope	2
3	Phase 1: Requirement Analysis & Planning	3
3.1	Understanding Business Requirements	3
3.2	Data Model Architecture	3
3.3	Object Tabs and Navigation Planning	4
4	Phase 2: Salesforce Development - Backend & Configurations	4
4.1	Declarative Automation: Record-Triggered Flows	4
4.2	Test Drive Reminder - End-to-End Verification	6
4.3	Programmatic Automation: Apex & Triggers	8
4.4	Inventory Validation - UI Testing	10
4.5	Stock Reduction after a Valid Confirmation	10
4.6	Apex Trigger Definition	13
4.7	Asynchronous Apex: Batch Processing	13
4.8	Scheduled Apex	14
4.9	Scheduling the Daily Batch Job	14
5	Phase 3: UI/UX Development & Customization	16
5.1	Lightning App & Tab Setup	16
5.2	Record Pages Used During Testing	16
6	Phase 4: Data Handling, Testing & Quality Assurance	16
6.1	Test Data Preparation	16
6.2	Functional Testing Matrix	17
6.3	Apex Test Coverage Scope	17
7	Phase 5: Deployment, Documentation & Maintenance	17
7.1	Deployment Readiness	17
7.2	System Monitoring & Troubleshooting	17
8	Implementation Architecture Summary	18
8.1	Completed Feature Summary	18
8.2	Future Enhancements	18
9	Conclusion	20

1 Project Overview

WhatsNext Vision Motors is implemented as a custom Salesforce CRM for managing vehicle, dealer, customer, order, test-drive, and service-request records. The project centralizes the core operational data required for automotive sales and follow-up activities while reducing repetitive manual work through automation.

Before automation, important tasks such as checking stock availability, assigning a dealer, and reminding customers about scheduled test drives can depend heavily on manual verification. The implemented Salesforce solution addresses these activities using a combination of declarative Flow automation and programmatic Apex logic. Vehicle Orders are validated against current stock, a pending order can be assigned to a dealer by matching stored location values, and scheduled test drives generate an automated email reminder.

The solution also includes asynchronous processing. Pending orders are eligible for later processing through Batch Apex when stock becomes available, while Scheduled Apex provides a recurring execution mechanism. This architecture demonstrates how Salesforce can combine data modelling, Lightning UI configuration, Flow Builder, Apex Triggers, Batch Apex, and scheduling in a single CRM implementation.

2 Objectives and Scope

The core goals of the project are directly related to improving customer handling, ensuring stock accuracy, and reducing manual operational effort.

2.1 Primary Business Objectives

- **Centralize Automotive CRM Data:** Maintain dedicated records for vehicles, dealers, customers, orders, test drives, and service requests.
- **Automate Dealer Assignment:** Create a Record-Triggered Flow that retrieves the customer's Address and finds a dealer whose Dealer Location matches that value.
- **Prevent Invalid Confirmed Orders:** Use Apex to stop a Vehicle Order from being saved as Confirmed when the related Vehicle has no stock.
- **Update Inventory Automatically:** Reduce Vehicle Stock Quantity when an order transitions to Confirmed and stock is available.
- **Improve Client Engagement:** Send an automated reminder one day before the Test Drive Date for records whose status is Scheduled.
- **Support Recurring Bulk Processing:** Use Batch Apex and Scheduled Apex to process pending orders automatically on a recurring schedule.

2.2 Project Scope

In-Scope: custom object creation, custom fields, lookup relationships, custom object tabs, a Lightning application, Record-Triggered Flows, email automation, Apex Trigger and Trigger Handler logic, Batch Apex, Schedulable Apex, scheduled-job verification, and UI-based functional testing.

Not Implemented in this project session: custom profiles and role hierarchy, advanced sharing rules, Dynamic Forms visibility rules, bulk data migration through the Data Import Wizard, duplicate rules, Change Set deployment, and dedicated Apex test classes. These topics appear in the reference report, but they are not claimed here because they were not part of the completed work.

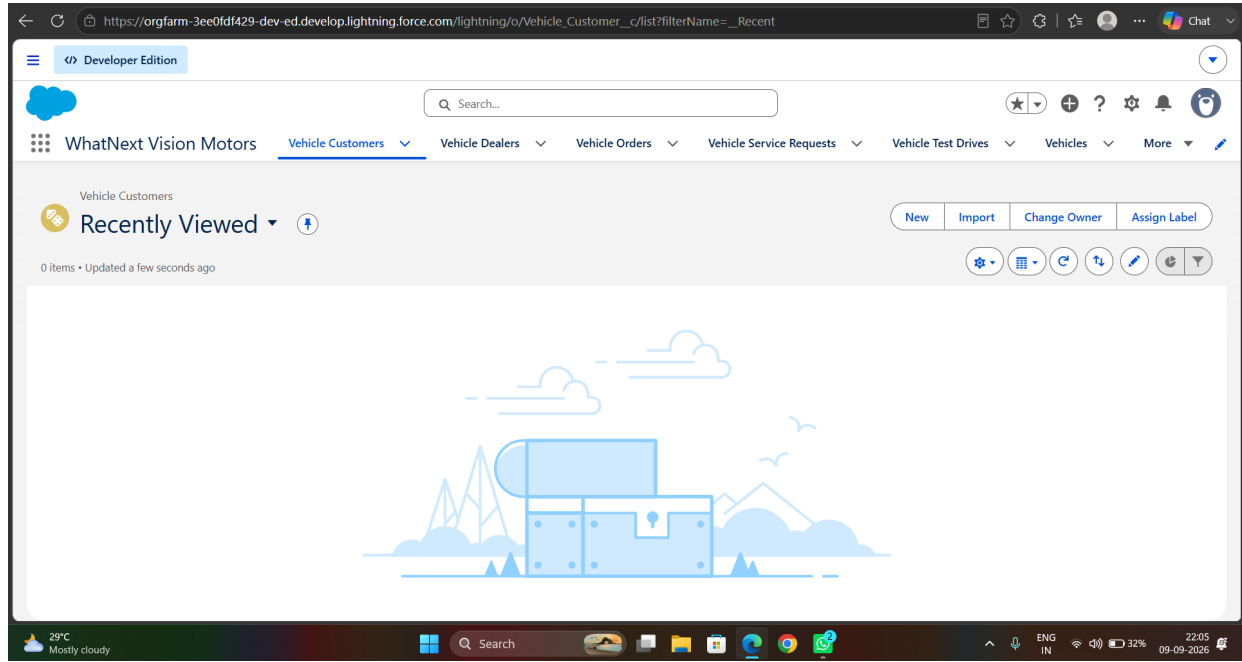


Figure 1: WhatsNext Vision Motors Lightning application showing the Vehicle Customers workspace

3 Phase 1: Requirement Analysis & Planning

3.1 Understanding Business Requirements

The project requirements were interpreted into a set of Salesforce features that could be implemented using both configuration and code. The customer, vehicle, dealer, order, test-drive, and service-request domains were separated into their own custom objects so each business entity could be maintained independently while still being related through lookup fields.

For orders, two business rules were especially important. A customer should not be able to complete a Confirmed order when the selected vehicle is out of stock, and the vehicle stock should be reduced after a valid confirmation. For pending orders, the design also required the ability to revisit records after stock replenishment. This naturally led to a combination of synchronous Trigger logic and asynchronous Batch Apex.

For test drives, the requirement was time-based rather than transaction-based. A scheduled test drive should cause a customer reminder to be sent before the appointment date. A Record-Triggered Flow with a Scheduled Path was therefore used instead of implementing the email logic in Apex.

3.2 Data Model Architecture

The implementation uses six main custom objects. Lookup relationships are used so records remain separately manageable while still supporting business navigation between objects.

Object Name	Key Custom Fields	Relationships
Vehicle__c	Vehicle Model, Stock Quantity, Price, Status	Lookup to Vehicle Dealer
Vehicle_Dealer__c	Dealer Location, Dealer Code, Phone, Email	Referenced by Vehicle and Vehicle Order
Vehicle_Customer__c	Customer Name, Email, Phone, Address, Preferred Vehicle Type	Referenced by Order, Test Drive, and Service Request
Vehicle_Order__c	Order Date, Status	Lookup to Vehicle Customer, Vehicle, and Vehicle Dealer
Vehicle_Test_Drive__c	Test Drive Date, Status	Lookup to Vehicle Customer and Vehicle
Vehicle_Service_Request__c	Service Date, Issue Description, Status	Lookup to Vehicle Customer and Vehicle

Table 1: Detailed Data Model Architecture and Field Specifications

The picklist values used in the project support common operational states. Vehicle Status includes Available, Out of Stock, and Discontinued. Vehicle Order Status includes Pending, Confirmed, Delivered, and Canceled. Test Drive Status includes Scheduled, Completed, and Canceled. Service Request Status includes Requested, In Progress, and Completed.

3.3 Object Tabs and Navigation Planning

The final application needed direct navigation to each primary custom object. Custom Object Tabs were therefore created and later added to the Lightning application navigation menu.

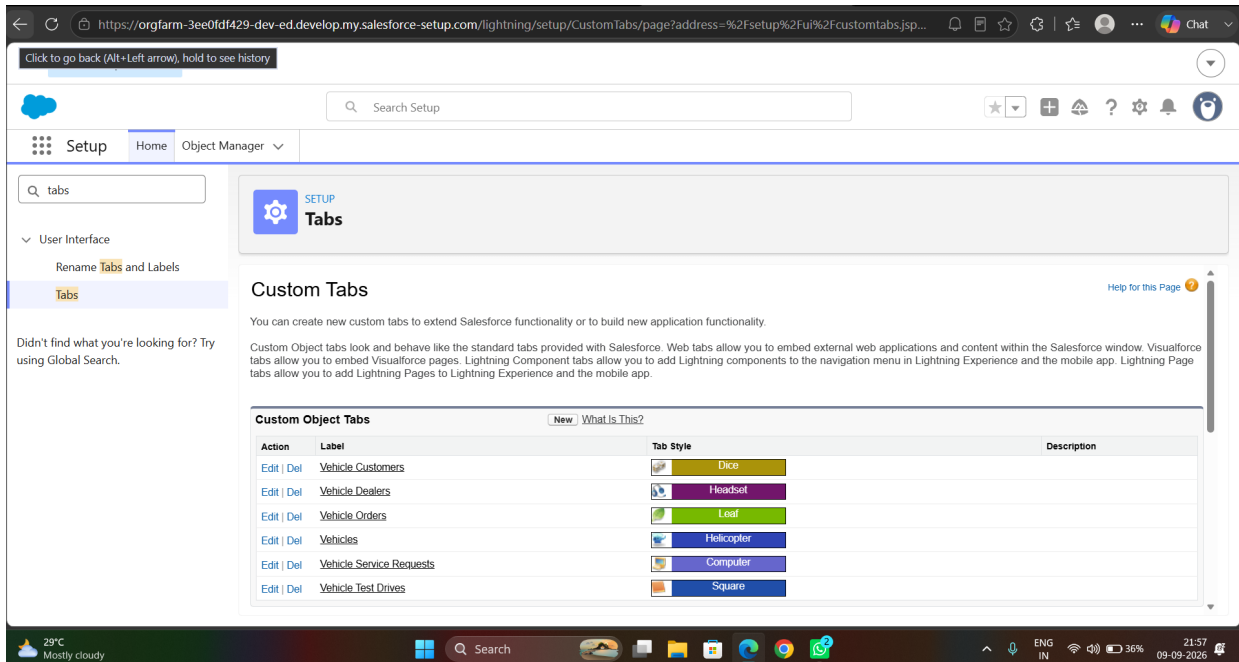


Figure 2: Custom Object Tabs created for the six project objects

4 Phase 2: Salesforce Development - Backend & Configurations

4.1 Declarative Automation: Record-Triggered Flows

Declarative automation was used where the business process could be expressed clearly with record queries, scheduled paths, and updates. Two flows were completed: *Auto Assign Dealer* and *Test Drive Reminder*.

Auto Assign Dealer Flow

The Auto Assign Dealer flow is triggered when a Vehicle Order is created with Status equal to Pending. The flow first retrieves the related Vehicle Customer using the lookup value stored on the triggering order. It then reads the Address field from that customer record. A second Get Records element searches Vehicle Dealer for a record whose Dealer Location matches the customer's Address. Finally, Update Records writes the matched dealer Record ID into the Dealer lookup on the triggering Vehicle Order.

This implementation uses an exact location-value match. Therefore, the word "nearest" in the flow label represents the project's simplified dealer-routing rule rather than a coordinate-based distance calculation.

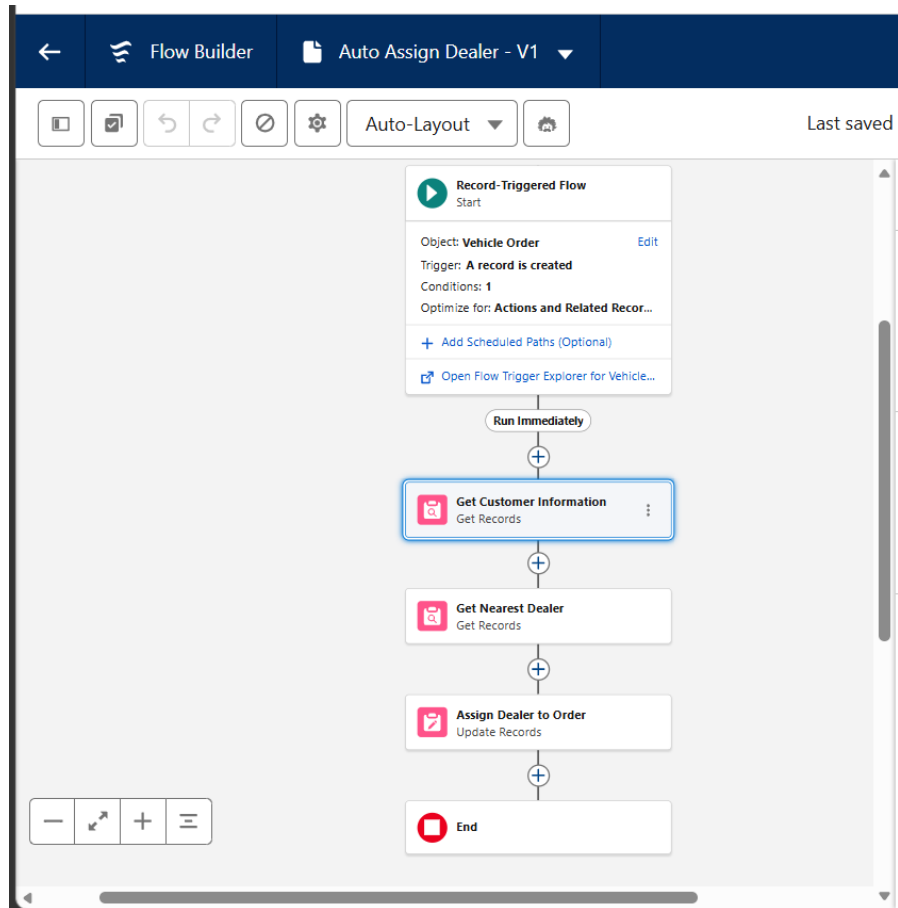


Figure 3: Auto Assign Dealer Flow configuration in Flow Builder

Test Drive Reminder Flow

The Test Drive Reminder flow is triggered when a Vehicle Test Drive record is created or updated and the Status is Scheduled. A Scheduled Path named *Reminder Before Test Drive* uses Test Drive Date as the time source and runs one day before that date.

The scheduled branch retrieves the related Vehicle Customer and uses the customer's Email field as the recipient address for the Send Email action. The subject configured during testing was "Reminder: Your Test Drive is Tomorrow!". The email body addresses the customer and reminds them that the test drive is scheduled for the following day.

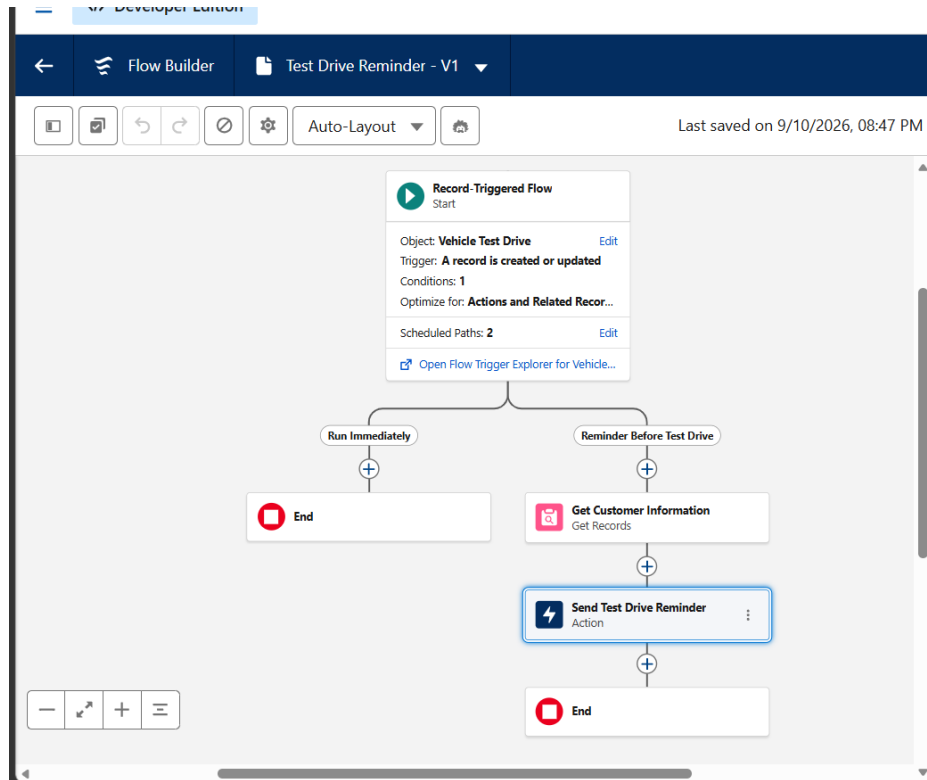


Figure 4: Test Drive Reminder Flow using a one-day-before Scheduled Path

4.2 Test Drive Reminder - End-to-End Verification

Functional verification was performed by creating a Vehicle Customer, creating a Vehicle Test Drive with Status set to Scheduled, and assigning a next-day Test Drive Date. The resulting test drive record was successfully stored in Salesforce and the reminder email was received in Gmail.

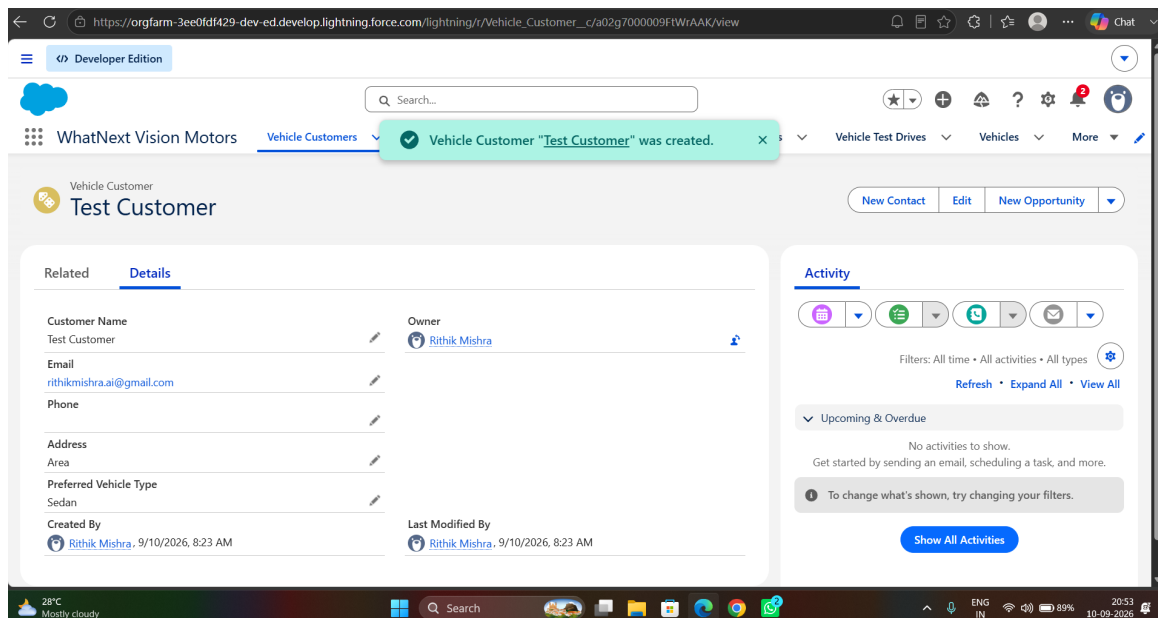


Figure 5: Vehicle Customer record used during test-drive reminder verification

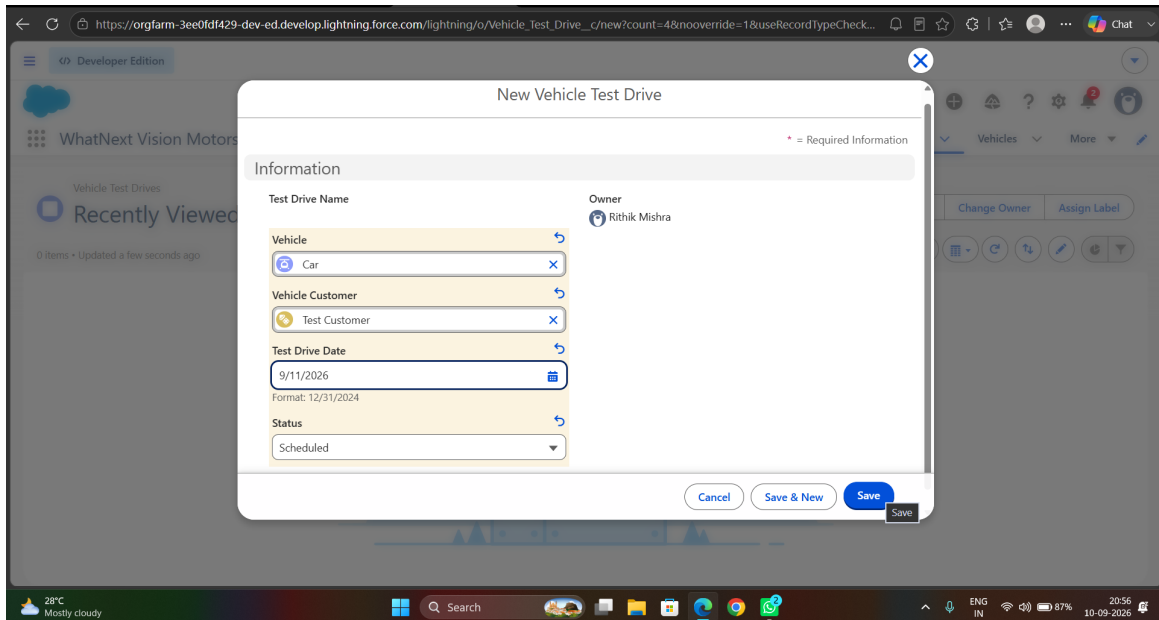


Figure 6: Creating a Vehicle Test Drive with Vehicle, Customer, Test Drive Date, and Scheduled Status

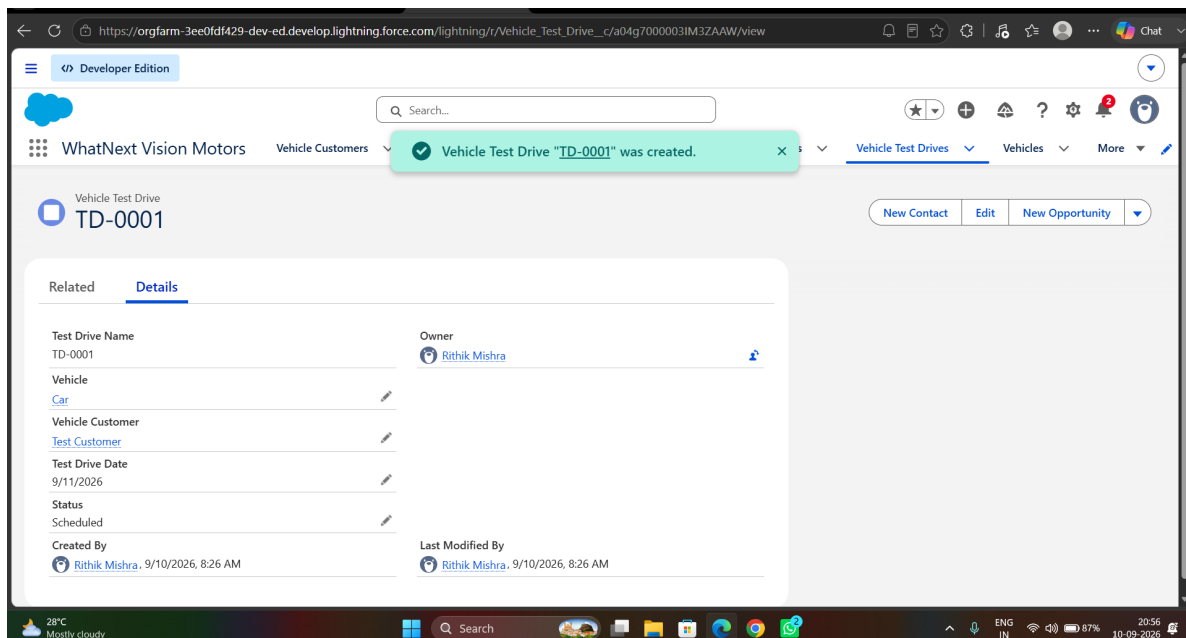


Figure 7: Vehicle Test Drive record created successfully in Salesforce

The email verification demonstrates that the related customer lookup, customer email retrieval, scheduled path, and Send Email action all worked together as intended.

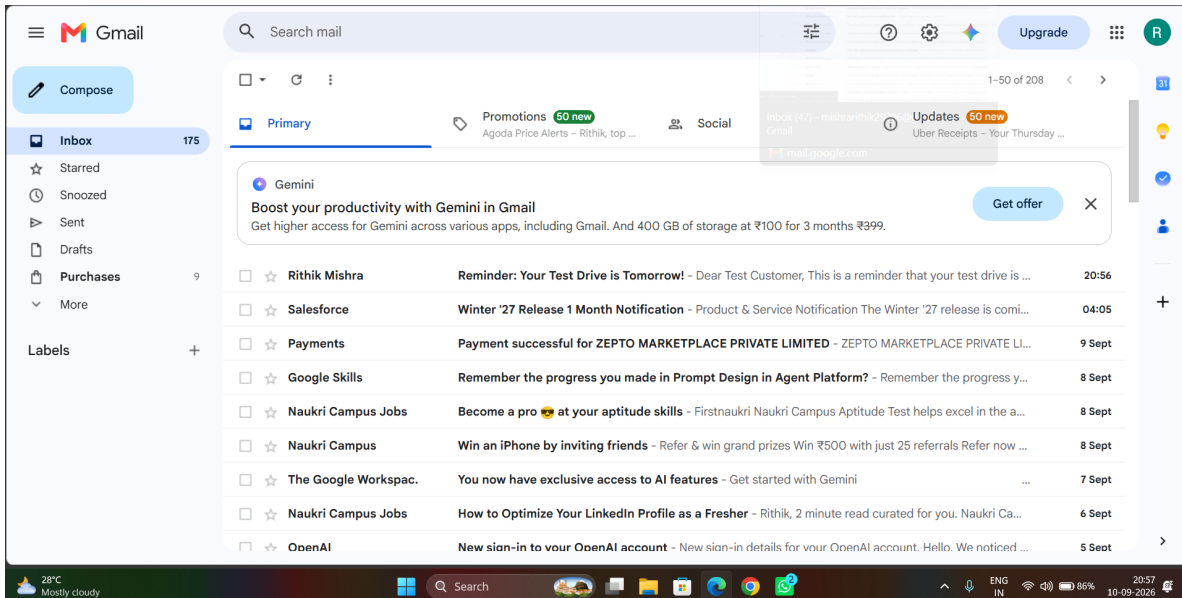


Figure 8: Gmail inbox showing the received Test Drive Reminder message

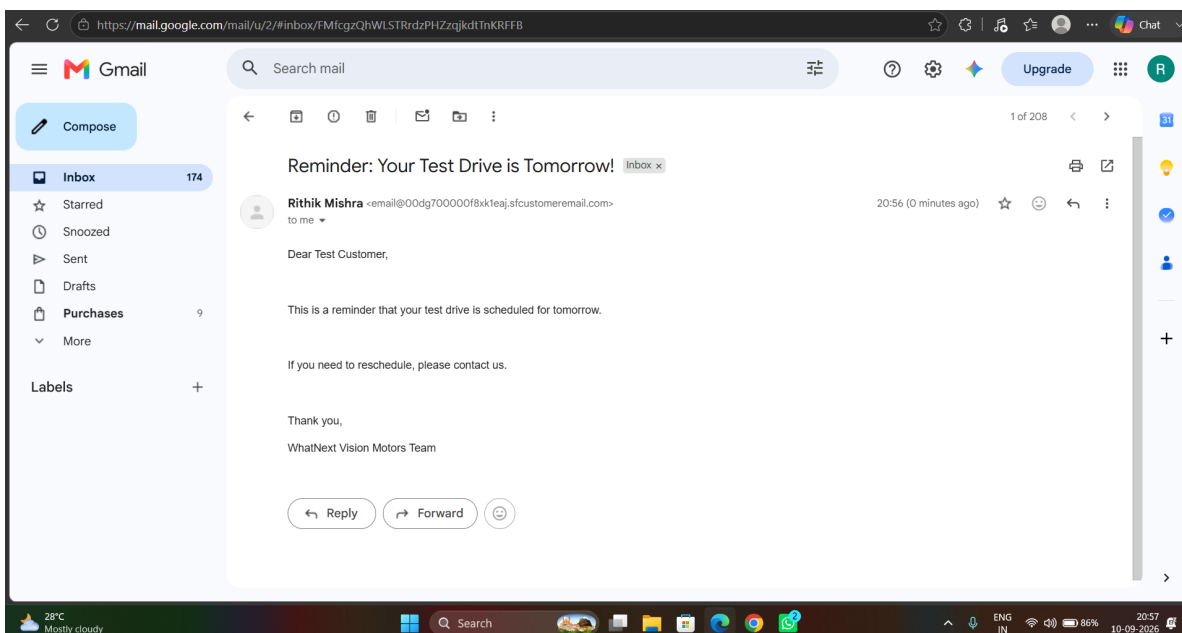


Figure 9: Opened Test Drive Reminder email received from Salesforce

4.3 Programmatic Automation: Apex & Triggers

Flows are appropriate for routing and scheduled communication, but stock validation must run directly inside the order transaction. Apex was used to enforce this rule so that a Confirmed order cannot be saved when the related vehicle does not have available stock.

The design follows a Trigger Handler pattern. The Trigger contains only the Salesforce trigger contexts and delegates all business logic to `VehicleOrderTriggerHandler`. This keeps the trigger small and makes the logic easier to maintain.

Trigger Handler Entry Logic

```
1 public class VehicleOrderTriggerHandler {
2     public static void handleTrigger(
3         List<Vehicle_Order__c> newOrders,
4         Map<Id, Vehicle_Order__c> oldOrders,
5         Boolean isBefore,
6         Boolean isAfter,
```

```

7     Boolean isInsert,
8     Boolean isUpdate
9 ) {
10     if (isBefore && (isInsert || isUpdate)) {
11         preventOrderIfOutOfStock(newOrders);
12     }
13
14     if (isAfter && (isInsert || isUpdate)) {
15         updateStockOnOrderPlacement(
16             newOrders, oldOrders, isInsert, isUpdate
17         );
18     }
19 }
20 }

```

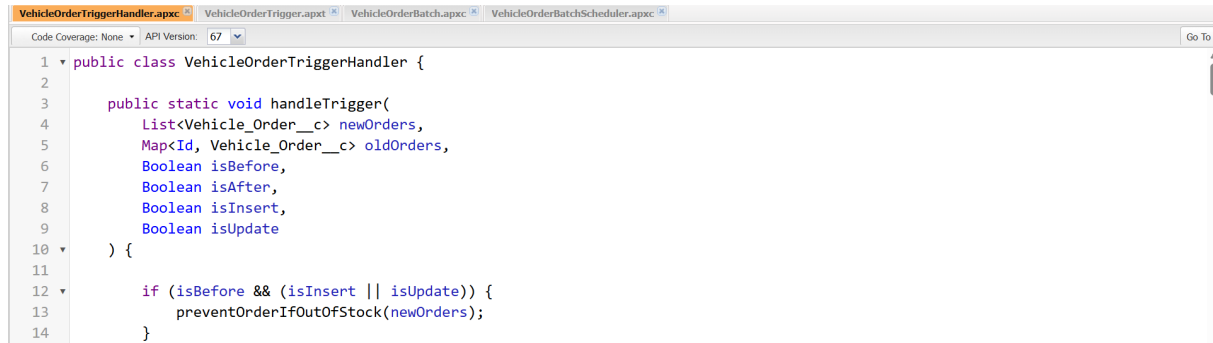


Figure 10: Developer Console showing the VehicleOrderTriggerHandler implementation

Out-of-Stock Validation Logic

The validation method first collects the Vehicle IDs referenced by relevant orders into a Set. A single SOQL query retrieves the stock quantities for those vehicles. For each order, if the vehicle's Stock Quantity is null, zero, or less than zero, `addError()` blocks the DML operation and displays a user-facing message.

```

1 private static void preventOrderIfOutOfStock(
2     List<Vehicle_Order__c> orders
3 ) {
4     Set<Id> vehicleIds = new Set<Id>();
5
6     for (Vehicle_Order__c orderRec : orders) {
7         if (orderRec.Vehicle__c != null &&
8             orderRec.Status__c == 'Confirmed') {
9             vehicleIds.add(orderRec.Vehicle__c);
10        }
11    }
12
13    Map<Id, Vehicle__c> vehicleStockMap =
14        new Map<Id, Vehicle__c>([
15        SELECT Id, Stock_Quantity__c
16        FROM Vehicle__c
17        WHERE Id IN :vehicleIds
18    ]);
19
20    for (Vehicle_Order__c orderRec : orders) {
21        Vehicle__c vehicle = vehicleStockMap.get(orderRec.Vehicle__c);
22        if (vehicle != null &&
23            (vehicle.Stock_Quantity__c == null ||
24             vehicle.Stock_Quantity__c <= 0)) {
25            orderRec.addError(
26                'This vehicle is out of stock. Order cannot be placed.'
27            );
28        }
29    }
30 }

```

4.4 Inventory Validation - UI Testing

The Apex rule was verified directly in the Salesforce UI. First, the test vehicle was configured with Stock Quantity equal to 0 and Status equal to Out of Stock.

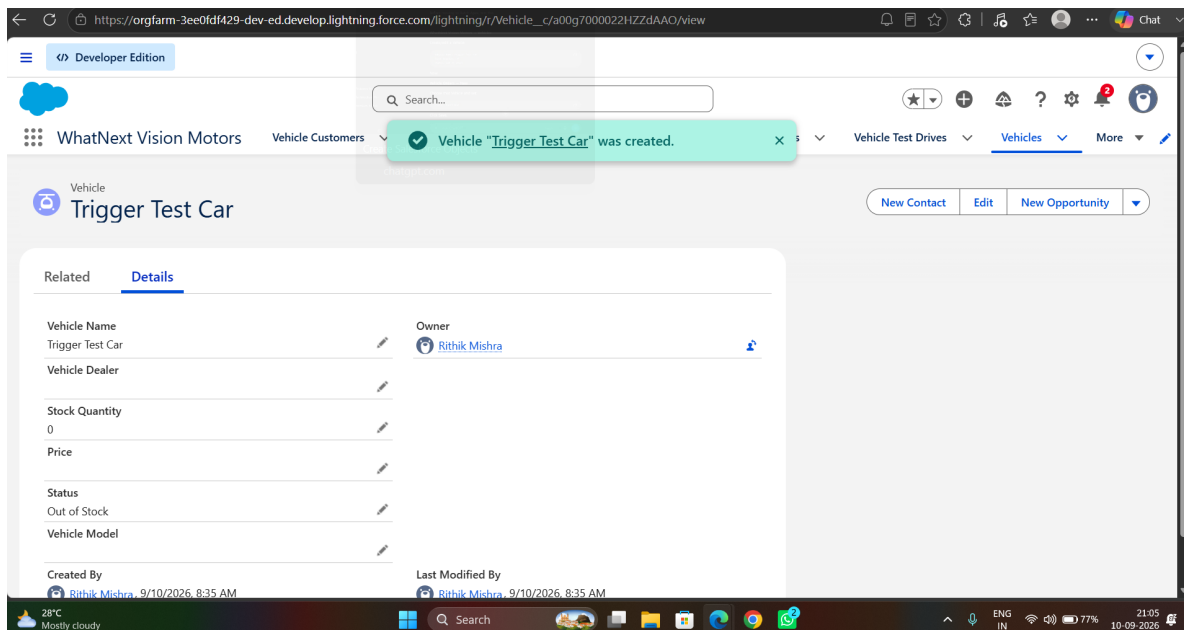


Figure 11: Vehicle record configured with Stock Quantity 0 and Out of Stock status

A new Vehicle Order was then created using that vehicle and the Status was set to Confirmed. When Save was clicked, the Trigger Handler stopped the transaction and displayed the configured custom error.

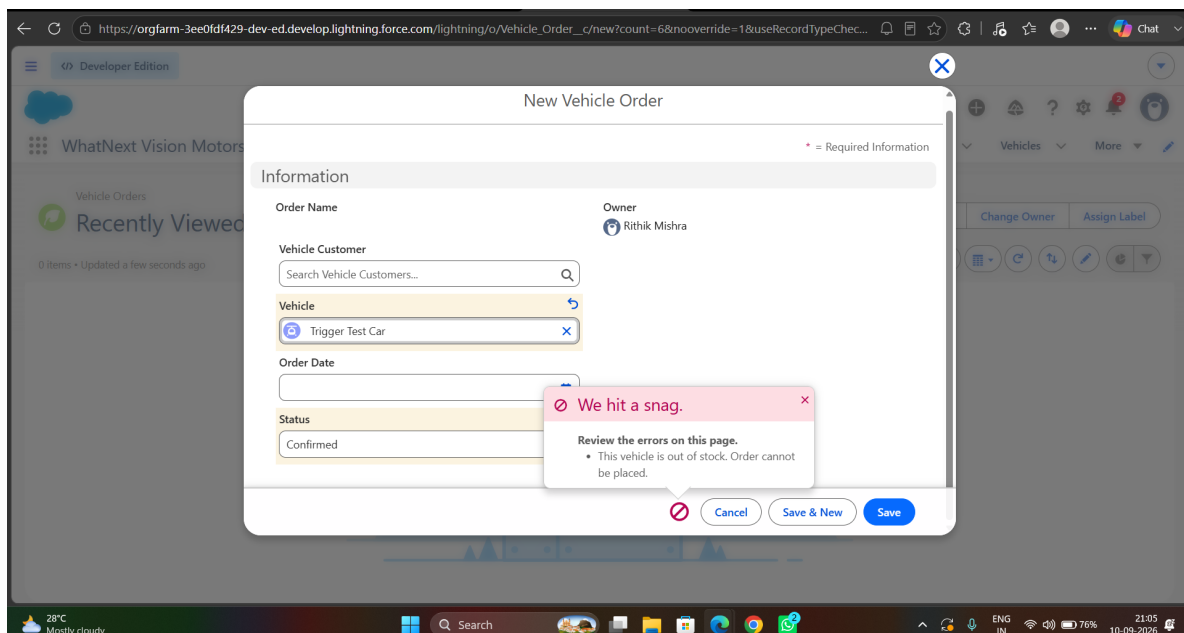


Figure 12: Out-of-stock Apex validation error displayed during Vehicle Order creation

4.5 Stock Reduction after a Valid Confirmation

A separate test was performed after making stock available. The same vehicle was edited so that stock was greater than zero. A Confirmed Vehicle Order could then be saved successfully. After confirmation, the Vehicle record reflected the remaining stock quantity.

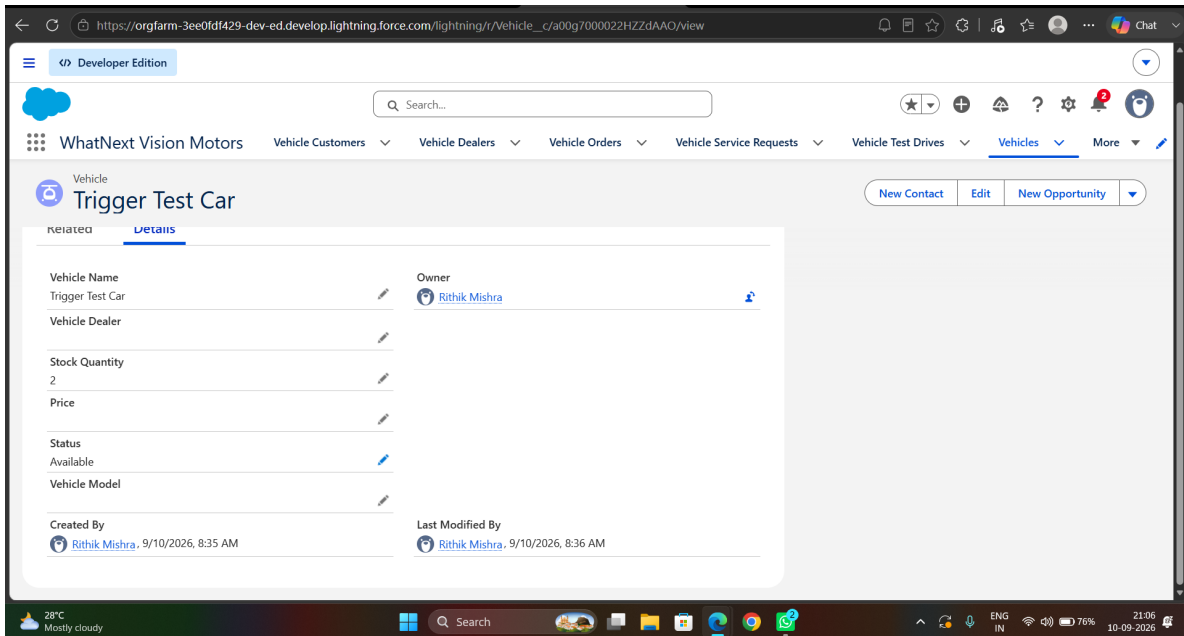


Figure 13: Vehicle record showing available stock before successful confirmation testing

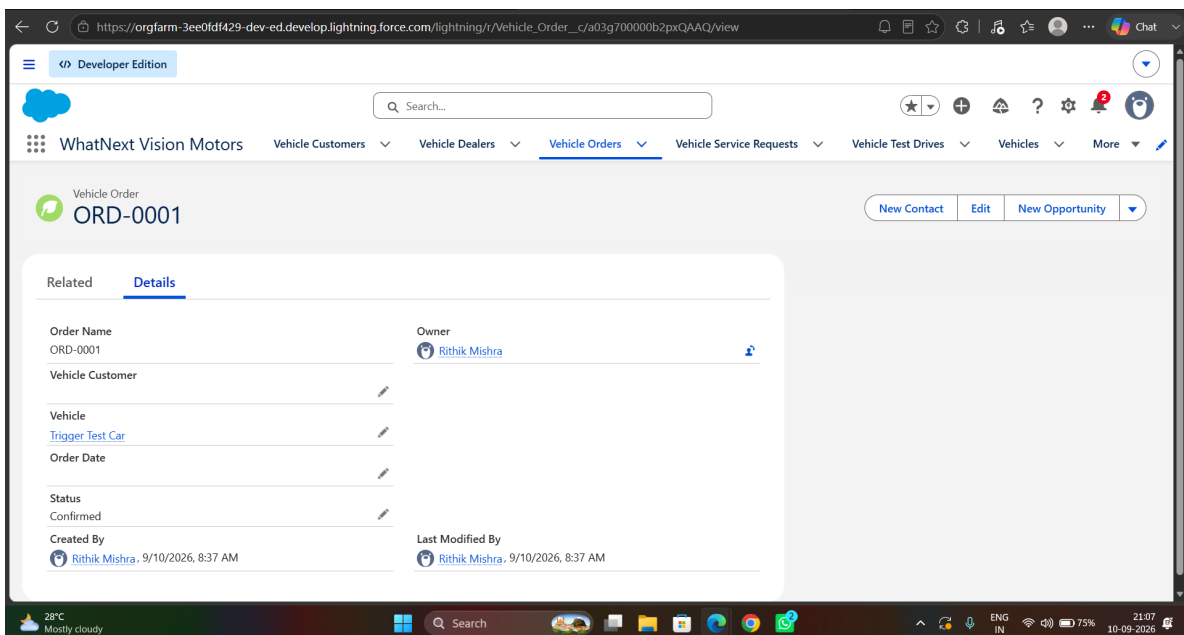


Figure 14: Vehicle Order record successfully saved with Confirmed status

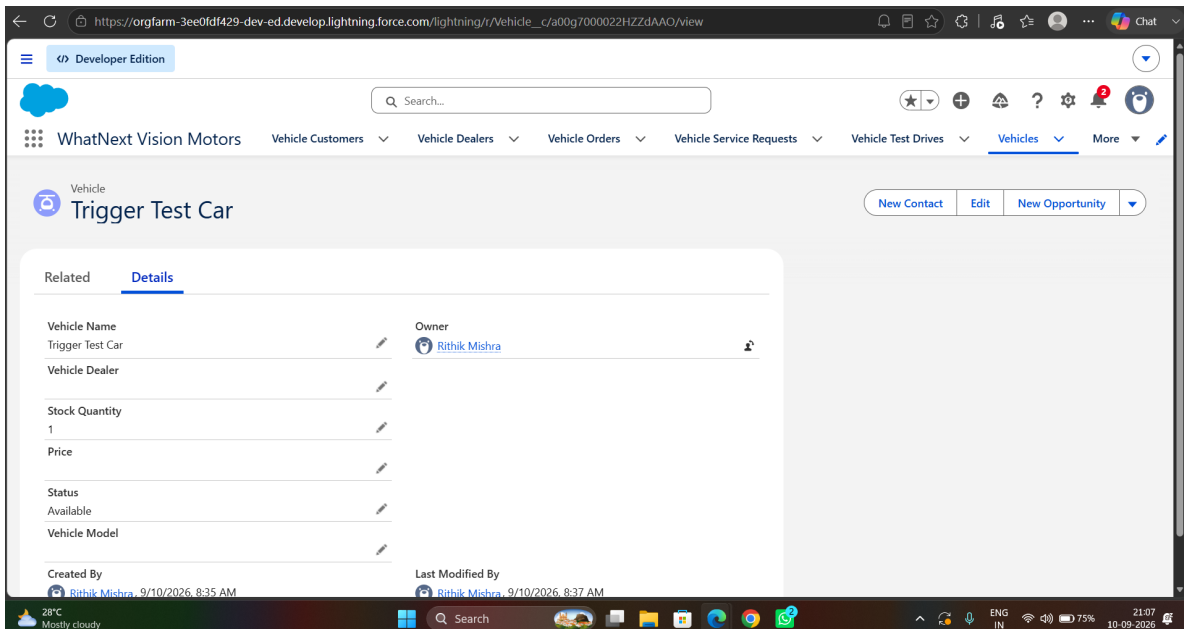


Figure 15: Vehicle record after confirmation showing the remaining Stock Quantity

4.6 Apex Trigger Definition

The trigger executes in before insert, before update, after insert, and after update contexts. The before contexts are used for validation because errors must be raised before the record is committed. The after contexts are used for related Vehicle updates after a successful order operation.

```
1 trigger VehicleOrderTrigger on Vehicle_Order__c (
2     before insert,
3     before update,
4     after insert,
5     after update
6 ) {
7     VehicleOrderTriggerHandler.handleTrigger(
8         Trigger.new,
9         Trigger.oldMap,
10        Trigger.isBefore,
11        Trigger.isAfter,
12        Trigger.isInsert,
13        Trigger.isUpdate
14    );
15 }
```

4.7 Asynchronous Apex: Batch Processing

Pending orders require a different execution model. The purpose of `VehicleOrderBatch` is to find Vehicle Order records whose Status is Pending and whose Vehicle lookup is populated. The batch checks whether stock is available and moves eligible orders to Confirmed. Once an order is changed to Confirmed, the existing Trigger logic handles the inventory reduction.

```
1 global class VehicleOrderBatch
2     implements Database.Batchable<sObject> {
3
4     global Database.QueryLocator start(
5         Database.BatchableContext bc
6     ) {
7         return Database.getQueryLocator([
8             SELECT Id, Status__c, Vehicle__c
9             FROM Vehicle_Order__c
10            WHERE Status__c = 'Pending'
11            AND Vehicle__c != null
12        ]);
13     }
14
15     global void execute(
16         Database.BatchableContext bc,
17         List<Vehicle_Order__c> orderList
18     ) {
19         Set<Id> vehicleIds = new Set<Id>();
20         for (Vehicle_Order__c orderRec : orderList) {
21             vehicleIds.add(orderRec.Vehicle__c);
22         }
23
24         // Query vehicle stock and update eligible pending orders.
25         // When order status becomes Confirmed, the trigger applies
26         // the related inventory update.
27     }
28
29     global void finish(Database.BatchableContext bc) {
30         System.debug('Vehicle order batch job completed.');
```

Batch Apex separates records into manageable execution scopes. This is more scalable than attempting to process every pending order in a single synchronous transaction, especially when the record volume increases.

4.8 Scheduled Apex

The batch is launched through `VehicleOrderBatchScheduler`, which implements the `Schedulable` interface. The scheduler creates a new batch instance and executes it with a batch size of 50.

```
1 global class VehicleOrderBatchScheduler implements Schedulable {
2     global void execute(SchedulableContext sc) {
3         VehicleOrderBatch batchJob = new VehicleOrderBatch();
4         Database.executeBatch(batchJob, 50);
5     }
6 }
```

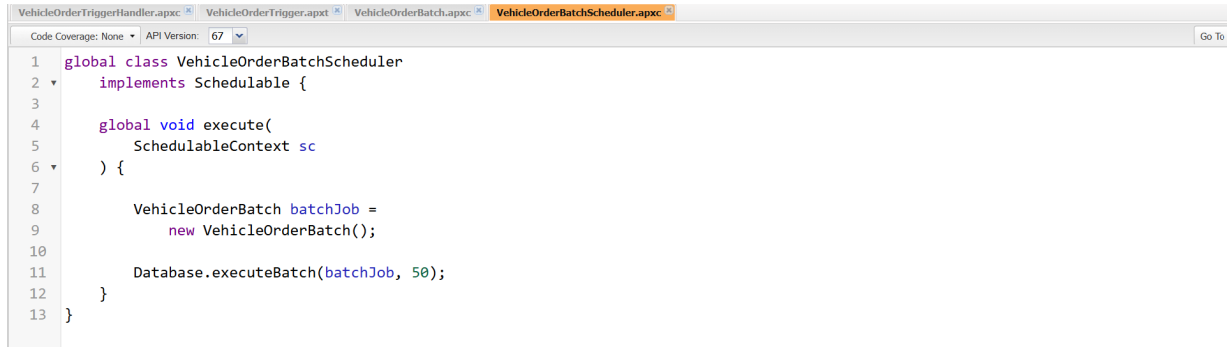


Figure 16: Developer Console showing the `VehicleOrderBatchScheduler` class

4.9 Scheduling the Daily Batch Job

The scheduler was registered through the Developer Console Execute Anonymous window. The final CRON expression used in the project was `0 0 0 * * ?`, which schedules execution every day at 12:00 AM (midnight).

```
1 String cronExp = '0 0 0 * * ?';
2 System.schedule(
3     'Daily Vehicle Order Processing',
4     cronExp,
5     new VehicleOrderBatchScheduler()
6 );
```

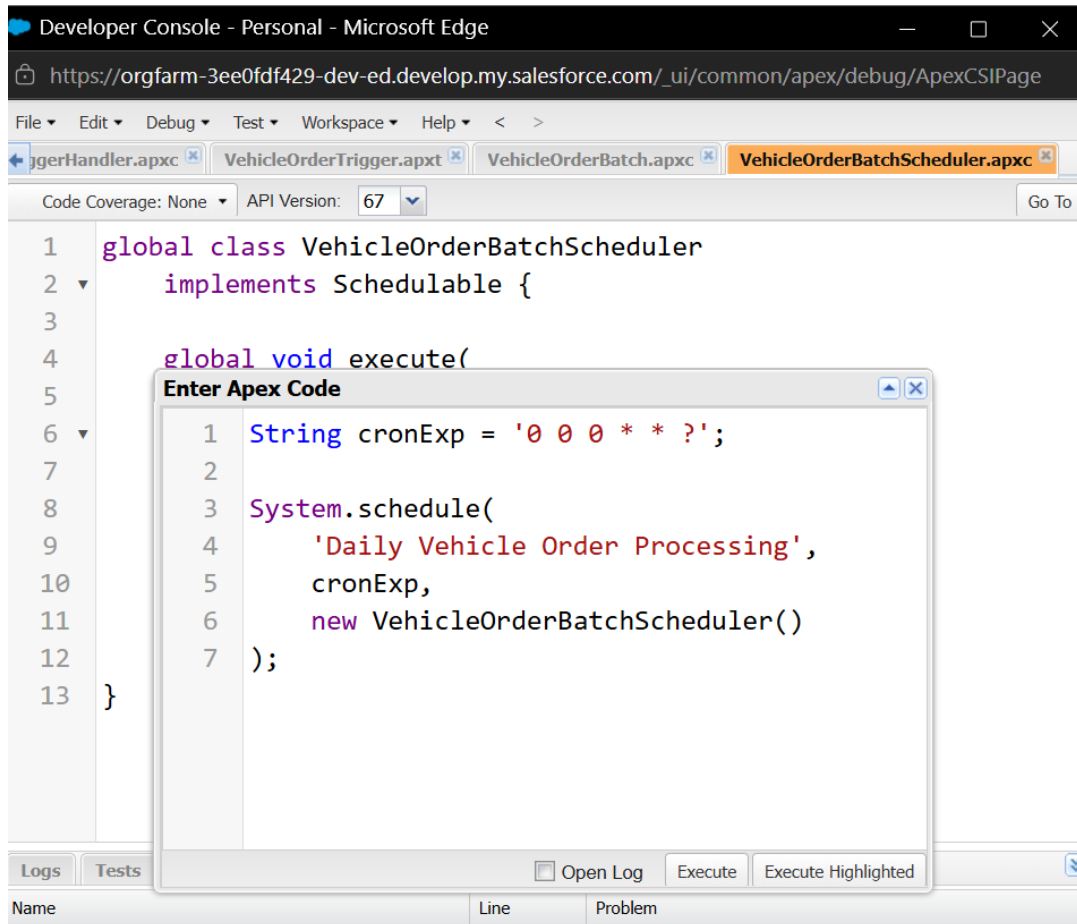


Figure 17: Execute Anonymous window used to register the daily Vehicle Order processing schedule

The job was then verified from Setup under Scheduled Jobs. The page showed *Daily Vehicle Order Processing* with Type equal to Scheduled Apex and the next scheduled run at 12:00 AM.

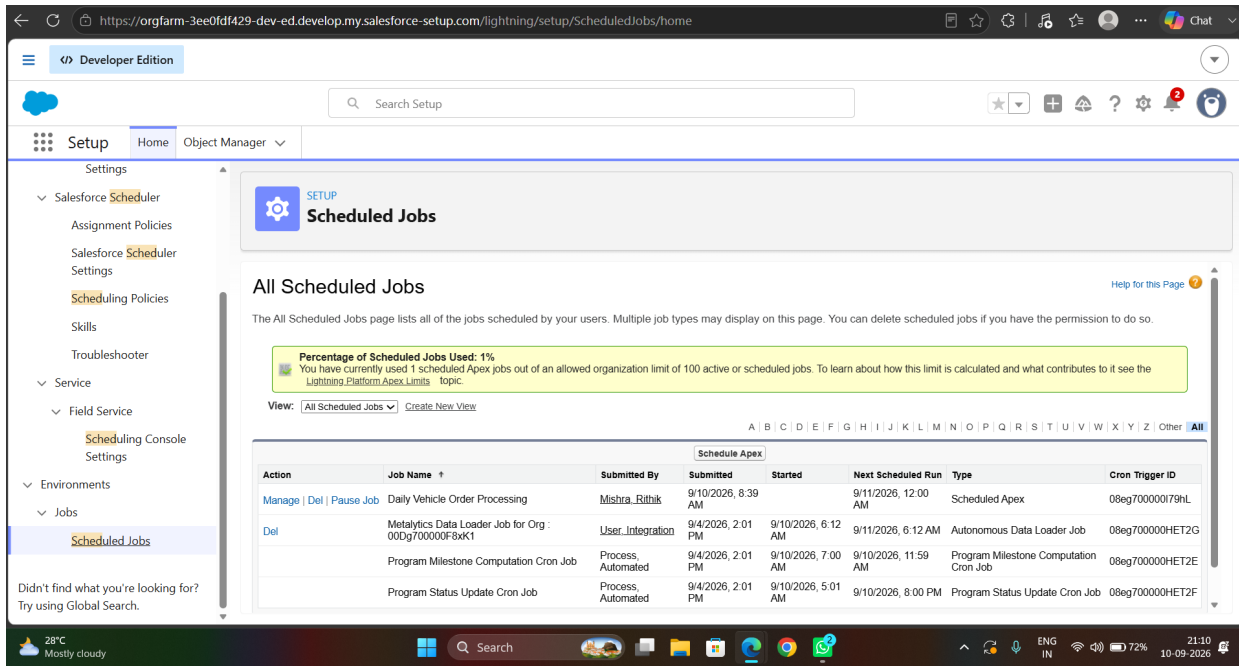


Figure 18: Scheduled Jobs page displaying Daily Vehicle Order Processing as Scheduled Apex

5 Phase 3: UI/UX Development & Customization

5.1 Lightning App & Tab Setup

The WhatsNext Vision Motors Lightning application serves as the main workspace for the implemented custom objects. The navigation contains Vehicle Customers, Vehicle Dealers, Vehicle Orders, Vehicle Service Requests, Vehicle Test Drives, and Vehicles, allowing direct movement between the key functional areas.

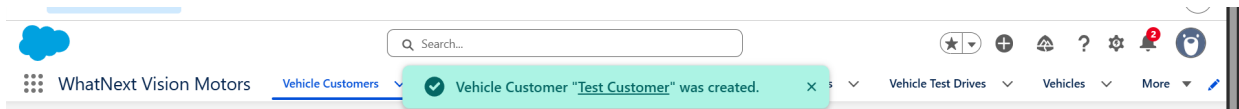


Figure 19: Lightning application navigation containing the main CRM modules

5.2 Record Pages Used During Testing

Standard Lightning record pages were used to review object details and perform functional testing. These pages exposed the fields required by the project, including stock quantity, order status, test-drive date, customer address, and related lookups. No separate Dynamic Forms visibility rules are claimed in this implementation.

The screenshots throughout this report demonstrate that the configured fields were available to users in the application and could be edited directly during validation and automation testing.

6 Phase 4: Data Handling, Testing & Quality Assurance

6.1 Test Data Preparation

The functional scenarios in this project were validated using records created directly through Salesforce Lightning. Test records included a customer, a vehicle used for multiple stock-level scenarios, a confirmed vehicle order, and a scheduled test drive.

Unlike the reference implementation, no bulk import process is claimed here. Test data was entered through the user interface because the main purpose of the project session was to validate the data model and automation behavior rather than perform a production migration.

6.2 Functional Testing Matrix

Component	Test Scenario	Observed / Expected Output	Status
Apex Trigger	Save a Confirmed order while Vehicle Stock Quantity = 0.	System displays “This vehicle is out of stock. Order cannot be placed.” and blocks the save.	Passed
Apex Trigger	Save a Confirmed order when Vehicle stock is available.	Order saves successfully and the vehicle stock quantity is reduced.	Passed
Record Flow	Create a Pending order with a related customer and matching dealer location.	Flow retrieves Customer, finds a matching Dealer, and updates the Dealer lookup on the order.	Configured
Record Flow	Create a Test Drive scheduled for the next day with Status = Scheduled.	Reminder email is sent to the Customer Email field and appears in Gmail.	Passed
Scheduled Apex	Register Daily Vehicle Order Processing using the scheduler class.	Scheduled Jobs lists the job as Scheduled Apex with next run at 12:00 AM.	Passed
Batch Apex	Configure processing of Pending orders when stock later becomes available.	Batch class is created and scheduled to evaluate Pending orders.	Configured

Table 2: System Testing and Quality Assurance Matrix

6.3 Apex Test Coverage Scope

The Developer Console screenshots show the Apex classes used by the implementation, but dedicated `@isTest` classes were not created during the completed work documented here. Therefore, this report does not claim a particular code-coverage percentage. The evidence presented is functional UI verification rather than deployment-level automated test coverage.

For production deployment, separate test classes should be added for the Trigger Handler, Batch Apex, and Scheduler so all important success and failure paths are covered and the Salesforce deployment coverage requirement is satisfied.

7 Phase 5: Deployment, Documentation & Maintenance

7.1 Deployment Readiness

The implementation was completed in a Salesforce Developer Edition org. A production deployment was not performed as part of this project session. If the solution is later moved to another org, the dependent metadata should be deployed together: custom objects and fields first, followed by tabs and app configuration, flows, Apex classes, and the trigger. After deployment, flows must be activated and the scheduled job must be registered in the destination org.

A recommended deployment sequence is:

1. Deploy custom objects, fields, lookup relationships, and picklist values.
2. Deploy custom tabs and Lightning app navigation configuration.
3. Deploy Flow definitions and activate the required versions.
4. Deploy the Trigger Handler, Trigger, Batch class, and Scheduler class together with test classes.
5. Recreate or schedule *Daily Vehicle Order Processing* in the target org.
6. Perform smoke tests for out-of-stock validation, stock reduction, dealer assignment, and reminder email delivery.

7.2 System Monitoring & Troubleshooting

The completed solution can be monitored using standard Salesforce Setup tools. Scheduled Jobs confirms whether the recurring scheduler is registered, while Apex Jobs can be used to inspect batch executions. Developer Console logs are useful when diagnosing trigger, batch, or scheduler behavior.

For Flow troubleshooting, Flow Builder Debug and Paused and Failed Flow Interviews can be used to investigate record retrieval, update, or scheduled-path issues. Email delivery problems should also be checked against org Deliverability settings and the recipient value returned by the customer lookup.

Typical troubleshooting checks include confirming that lookup fields contain valid IDs, picklist values match the Flow entry conditions exactly, Customer Address and Dealer Location contain matching text for dealer assignment, and Vehicle Stock Quantity contains the value expected by the Apex validation logic.

8 Implementation Architecture Summary

The final implementation can be summarized as a set of connected synchronous and asynchronous processes.

- **Data Layer:** Six custom objects store vehicles, dealers, customers, orders, test drives, and service requests.
- **Navigation Layer:** Custom tabs and the WhatsNext Vision Motors Lightning app expose the objects to users.
- **Dealer Assignment:** A Record-Triggered Flow runs for newly created Pending orders and populates the Dealer lookup through a location match.
- **Test Drive Communication:** A Scheduled Path runs one day before Test Drive Date and sends a reminder email to the related customer.
- **Order Validation:** The Apex Trigger Handler blocks Confirmed orders when the vehicle has no stock.
- **Inventory Update:** When an order becomes Confirmed and stock is available, the related Vehicle stock is reduced.
- **Bulk Processing:** VehicleOrderBatch evaluates Pending orders and confirms eligible records after stock becomes available.
- **Recurring Execution:** VehicleOrderBatchScheduler launches the batch on the registered daily schedule.

This layered approach separates user-interface configuration, declarative automation, transactional logic, and asynchronous processing so each part of the solution has a clear responsibility.

8.1 Completed Feature Summary

The completed CRM can be viewed as four cooperating layers. The data layer stores the core automotive entities and their relationships. The user-interface layer exposes those records through the Lightning application and custom tabs. The automation layer contains the two Record-Triggered Flows used for dealer assignment and test-drive communication. The programmatic layer contains the Trigger Handler, Trigger, Batch Apex, and Scheduler classes used for inventory integrity and recurring processing.

The functional evidence captured during implementation confirms the most important user-facing outcomes: a scheduled test drive generated a real reminder email, a confirmed order was blocked when stock was zero, a valid confirmed order was saved when stock was available, and the daily scheduler appeared correctly in Salesforce Scheduled Jobs. These outcomes connect the configuration work to visible business behavior rather than documenting setup screens alone.

8.2 Future Enhancements

The current project provides a solid base for further development. A more advanced dealer-assignment mechanism could replace exact text matching with latitude/longitude distance calculations. Dedicated Apex test classes should be added to provide deployment-ready automated coverage for all success and failure paths. Additional inventory controls could automatically change Vehicle Status to Out of Stock when quantity reaches zero and return it to Available after replenishment.

The service-request module can also be extended with technician assignment, service priority, estimated completion date, and customer notifications. Reporting dashboards could summarize available inventory, pending orders, test-drive conversion, and service-request volume. If the solution later in-

tegrates with an external inventory or ERP platform, scheduled stock updates could be replaced or supplemented by API-driven synchronization.

9 Conclusion

The WhatsNext Vision Motors CRM demonstrates an end-to-end Salesforce implementation focused on practical automotive sales and customer-service workflows. The project establishes a structured custom data model, exposes it through a dedicated Lightning application, and automates key processes using Record-Triggered Flows and Apex.

The Auto Assign Dealer flow reduces manual dealer routing by retrieving the customer location and assigning a matching dealer to pending orders. The Test Drive Reminder flow successfully delivered a scheduled email to the customer. Programmatic stock validation prevented an out-of-stock vehicle from being confirmed, while valid orders were saved when inventory was available and the stock quantity was updated. Batch Apex and Scheduled Apex extend the solution beyond individual transactions by providing recurring processing for pending orders.

The screenshots and testing evidence included throughout this document correspond to the actual work performed in the Salesforce org. Features present only in the supplied reference report but not implemented in this project session have been identified as outside the completed scope rather than being presented as completed work. This keeps the documentation aligned with the real implementation while preserving the report structure, section hierarchy, figure placement, and technical depth of the reference document.

From a Salesforce development perspective, the project demonstrates several important design practices: keeping the trigger thin by delegating logic to a handler class, bulkifying record queries with Sets and Maps, using Flow for administrator-friendly automation, using Batch Apex for recurring multi-record work, and using Scheduled Apex only as the timing mechanism that launches the batch. These design choices make the implementation easier to understand and provide a clear path for future extension.

Final Project Outcomes

- Six custom objects and their key fields and lookup relationships were created.
- Custom tabs and a dedicated Lightning application were configured.
- Auto Assign Dealer and Test Drive Reminder Record-Triggered Flows were built.
- The Test Drive Reminder was verified through an actual received email.
- An Apex Trigger Handler blocked out-of-stock confirmed orders and supported stock updates.
- Batch Apex and a Schedulable class were created for recurring pending-order processing.
- The daily processing job was successfully registered and verified in Scheduled Jobs.

The result is a coherent Salesforce CRM prototype in which data modelling, UI configuration, declarative automation, synchronous Apex, and asynchronous Apex work together around a single automotive business process.